

Ignition Perspective

A hands-on curriculum for Perspective Designer

Ignition **8.3.9** · Inductive Automation

How Perspective actually works — the Gateway/Designer/Session model, views and containers, bindings and transforms, tags and UDTs, scripting, security and alarms — followed by a seven-day training plan, eighteen hands-on labs, and a capstone project.

18 labs **7**-day plan **1** capstone project **18** reference parts

Prepared September 15, 2026

Read This Part, At Least

You've just logged into Perspective Designer for the first time. Right now it looks like a cockpit with too many switches and no labels. That feeling is correct and it is temporary.

Here's the deal I'm offering: Perspective is not a big product pretending to be small. It's a **small** product pretending to be big. There are maybe nine real ideas in it. Once you have those nine, the other four hundred menu items are just places where the nine ideas live.

How to read this when your focus is shot

I wrote this for someone who reads in twelve-minute windows and gets interrupted. So:

If you have	Do this
5 minutes	Read The Whole Product in Ten Bullets . That's it. You'll be less lost than half the people on the forums.
15 minutes	One numbered section (they're 3–5 minutes each). Stop at the end of one. Don't power through.
An hour	One Part, then its lab. Hands-on resets attention better than coffee.
You're fading	Stop reading and go do a lab. Fading brain + reading = nothing retained. Fading brain + clicking = something retained.

Four things are load-bearing. Please don't skip them:

1. **Every Part opens with "The one thing."** One sentence. If you read only those eighteen sentences, you have the product.
2. **Every Part closes with a  Recall box.** Two or three questions, answers on the line below. Cover the answers with your hand, guess, then look. This is not filler — *trying and failing to remember* is what makes it stick. Reading it again does almost nothing. Skipping these costs you more than skipping whole sections.
3. ** marks a trap.** Every one of these is something I have personally watched eat an afternoon. They are the most valuable paragraphs here.
4. ** marks a law** — a named idea you can carry around. There are about a dozen. Learn the names; the names are the handles.

The legend

Mark	Means
	A law . Named so you can recall it later.
	A trap . Costs an afternoon.
	Recall . Try before you look.
	Roughly how long a Part takes.
	Boss fight — how you know a lab is actually done.
FIELD NOTE	A story. Stories are how humans store rules.

Table of Contents

Read This Part, At Least	2
The Whole Product in Ten Bullets	4
Part 1. The Server Has the Brain	5
Part 2. The Cockpit	9
Part 3. Views, and the Art of Not Drawing Pixels	11
Part 4. Bindings: Where the Magic Actually Is	14
Part 5. Scripts, Events, and Passing Notes	18
Part 6. Tags, History, and Not Getting Fired by SQL	22
Part 7. Pages, Docks, and Popups	25
Part 8. Style Without Regret	28
Part 9. Security That Means Something	30
Part 10. Alarms	32
Part 11. What's New in 8.3	33
Part 12. The Seven-Day Plan	34
Part 13. The Labs	37
Part 14. When It Breaks	50
Part 15. The Rules	52
Part 16. Cheat Sheets	54
Part 17. Glossary	57
Part 18. Final Exam	58

The Whole Product in Ten Bullets

Read this once now. Read it again in three days. It's the map everything else hangs on.

1. **The Gateway is a server** that talks to PLCs, talks to databases, holds every live value, and runs every line of your code.
2. **The Designer is a desk.** You sit at it and file paperwork. The Gateway acts on that paperwork immediately — there's no build, no deploy.
3. **A Session is a window.** It runs in a browser. It has no brain. It draws what the Gateway sends and reports back when somebody touches something.
4. **A tag is a named value** with a quality and a timestamp. Tags are the nouns of the whole system.
5. **A UDT is a tag type** — define "Motor" once, stamp two hundred of them. This is the single biggest lever in the product.
6. **A View is a reusable screen-piece** that takes parameters, like a function.
7. **A binding is a live wire** from data to a property. Bindings, not scripts, are how Perspective is meant to be built.
8. **Containers decide layout.** Flex for almost everything; Coordinate when you're drawing a picture of a physical thing.
9. **Named Queries are how you touch SQL.** Never string-concatenate SQL. Never.
10. **Security lives on the tag and the query**, not on the button you hid.

That's the product. Everything below is detail, war stories, and where the bodies are buried.

Part 1. The Server Has the Brain

 20 minutes · **The one thing:** in Perspective the browser is a screen, not a computer — the Gateway holds every value and runs every line of your code.

Every beginner asks the same question in week two, usually with a slightly wounded tone: "Why can't I just save a file to the operator's laptop?" The answer is one sentence long and it also explains about forty other things. Let's go earn it.

1.1 Three things are called "Ignition" and nobody tells you which

The Gateway is the server. The brain. A Java service running on some machine in a rack, which:

- talks **down** to equipment — OPC UA to PLCs, Modbus, drivers, MQTT
- talks **sideways** to databases over JDBC
- holds the **tag system**, the live in-memory picture of your plant
- **runs all project logic** — bindings, scripts, queries, alarms
- and **serves** the web pages

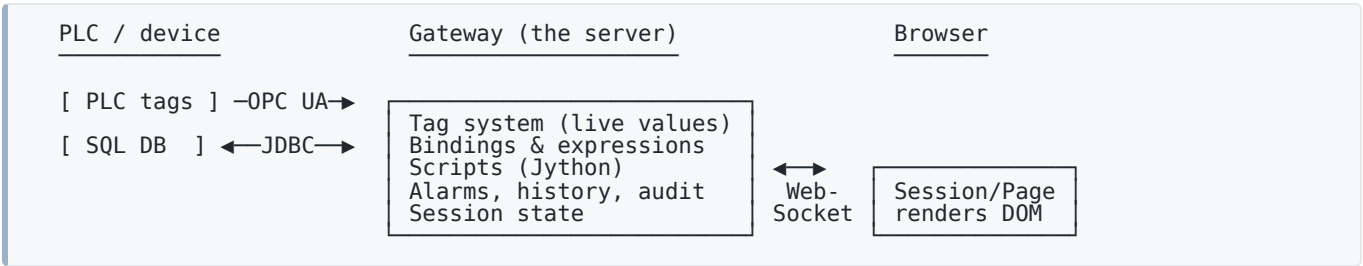
You administer it in a browser at `http://<gateway-host>:8088` (or `8043` for HTTPS).

The Designer is the desk you sit at. Launch it from the Designer Launcher; it connects to the Gateway and edits the project *live on the server*. There is no compile. There is no deploy. You press save and the running system has changed. This is either exhilarating or terrifying depending on which Gateway you're pointed at.

A Perspective Session is what the operator sees: your project in a browser tab or the Perspective mobile app. Note the word *window*. It has no opinions of its own.

1.2 The law that explains everything else

 **The Law of the Server's Brain.** The browser is a screen, not a computer. The Gateway holds the state and runs the logic. The two talk over a persistent WebSocket. You click a button, the click travels to the Gateway, the Gateway runs your Python and changes some properties, and the changed properties travel back down and the browser redraws.



Three consequences you will run into this week:

- **Your Python runs on the server.** So no, you can't open a file on the operator's laptop, and `print` doesn't go to the browser console. It goes to the Gateway.

- **Every bound property is a subscription the Gateway maintains — per session.** One heavy screen × five hundred operators is five hundred times the work. Screen design is a performance decision, which is a sentence nobody tells you until the day it matters.
- **A session survives a bad network better than you'd guess**, but it isn't offline-capable by default. (8.3 adds specific offline *form* support — see [Part 11](#).)

1.3 Perspective or Vision: which world am I in?

Your Gateway probably has both modules installed, which is a great way to follow a tutorial for forty minutes before realizing it was written for the other one.

	Perspective	Vision
Runs in	Web browser, phone, tablet	Java desktop client
Layout	Flex, breakpoints, responsive	Fixed pixels and anchors
Your scripts run	On the Gateway. Always.	On the operator's machine
Unit of design	View	Window
Mobile	Yes, properly	No
Learn it if	You're starting today	You inherited it

Learn Perspective. Vision is supported and plenty of plants run on it happily, but new work goes to Perspective.

1.4 Tags: the nouns

A **tag** is a named value living in the Gateway, carrying a value, a **quality**, a **timestamp**, a datatype, and optionally alarms and history settings.

Tags live in a **provider**. You get `default` (your real tags) and `System` (diagnostics). Paths look like `[default]Line1/Tank1/Level` and `[System]Gateway/CurrentDateTime`.

Type	What it is	You'll use it for
OPC	Wired to a device address	The actual PLC value
Memory	Value lives only in Ignition	Setpoints, simulation, scratch
Expression	Computed, re-evaluates on change	<code>{[.]Level} / {[.]Capacity} * 100</code>
Query	Computed by SQL on a poll	Slow business data
Derived	Read/write transform over a source	Unit conversion that writes back
Reference	A pointer to another tag	Aliasing during reorganization
UDT Definition	A tag <i>type</i>	"Motor", "Tank", "Valve"
UDT Instance	One of that type	<code>Line1/Pump3</code>

 **The 200-Motor Test.** Before you build anything twice, ask: *would this survive two hundred motors?* Define a UDT called `Motor` once — `Running`, `Fault`, `Speed`, `Hours`, its alarms, its history — then stamp two hundred instances. Change the definition, all two hundred change. Pair that with a View that takes a tag path as a parameter and you build **one** motor faceplate for the whole plant. Every experienced Ignition developer thinks in UDTs. It is the difference between a week of work and an afternoon.

About quality. A tag value is never just a number — it carries quality (`Good`, `Bad_NotFound`, `Bad_Stale`, `Uncertain`...). Perspective draws a **quality overlay** on components bound to a bad tag. That hatched or marked-up component you'll eventually see is not a rendering bug. It's Ignition refusing to lie to your operator about whether the number is real.

⚠ Never globally suppress quality overlays. I've seen a control room stare at a frozen number for two shifts because someone found the opt-out checkbox and liked how clean it looked.

1.5 The lifecycle of one click

Memorize this. It answers most "why isn't this working" questions before you have to ask them.

1. Operator clicks a Button in the browser.
2. The click goes over the WebSocket to the Gateway.
3. The Gateway runs your `onActionPerformed` script, in Jython, **on the server**.
4. Your script writes a tag with `system.tag.writeBlocking(...)`.
5. The tag system pushes it to the PLC.
6. The PLC value changes; the tag subscription fires.
7. Every binding on that tag, in every open session, recomputes.
8. Changed properties stream down; browsers redraw.

Steps 4 through 7 are a round trip through actual copper.

⚠ **The Stale Read.** You write a tag, read it back on the next line, get the old value. So you write it again. Then you add a delay. Then you begin quietly questioning your career choices. The value hasn't come back yet — it's somewhere between here and a PLC. **Don't read after write.** Write, and let the binding tell you what happened. If you need confirmation that the write was *accepted*, check `result[0].good`. That's a different question from "did the value change," and knowing the difference is most of the skill.

FIELD NOTE The stale read is the single most common bug in new Perspective code, and it almost never gets diagnosed as itself. It gets diagnosed as "Ignition is slow," "the PLC is flaky," or "this tag is broken." I once watched two engineers add retry loops to four separate screens over a week. The fix was deleting eleven lines.

RECALL Cover the answers.

1. Where does a Perspective event script execute?
2. What's the name of the trap where you write a tag and read back the old value?

3. Why is screen design a performance decision?

Answers: (1) on the Gateway, always. (2) The Stale Read. (3) every bound property is a subscription the Gateway maintains per session, so cost multiplies by the number of open sessions.

Part 2. The Cockpit

 15 minutes · **The one thing:** the Property Editor's four tabs — PROPS, POSITION, META, CUSTOM — are the whole API, and CUSTOM is where good architecture lives.

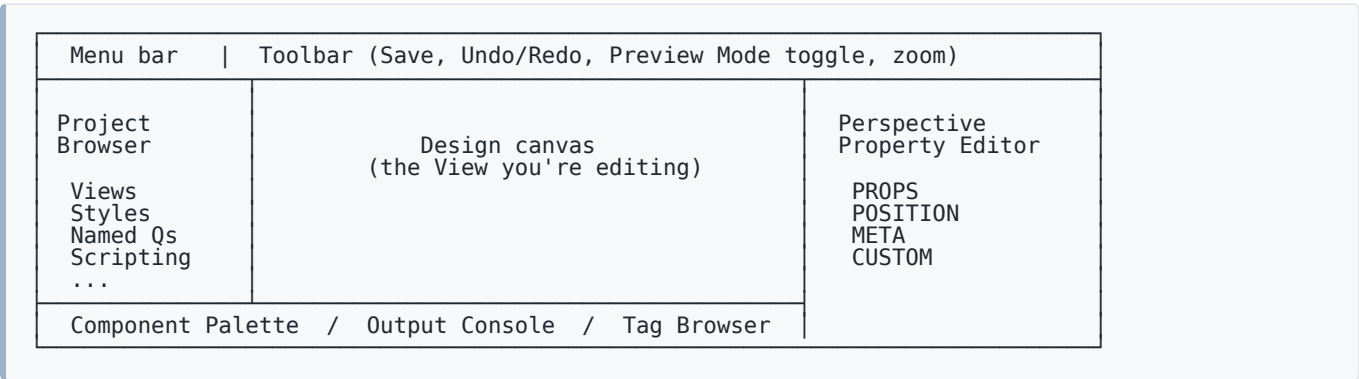
2.1 Getting in

1. From the Gateway home page, install the **Designer Launcher** (once per machine).
2. Add your Gateway by hostname or IP. It shows up as a tile.
3. Launch, log in with a user holding the **Designer** role, pick your project.

Designer sessions are collaborative — several people can be in one project, and Perspective resources **lock** to whoever opened them for editing. If you see a resource marked in use by someone else, that's the concurrency system doing its job, not a fault.

 Install the *Designer* Launcher, not the *Vision Client* Launcher. They're different downloads with similar names on the same page, and about a third of new people get this wrong on day one.

2.2 The panels



Project Browser (left) — the tree of everything. **Perspective > Views** is where you'll live. When a view is open it also shows that view's **component hierarchy**, which is how you select a component you can't click: too small, hidden, or buried under something else.

Design canvas (center) — two modes, toggled in the toolbar:

- **Design mode** — clicks select things for editing
- **Preview mode** — clicks behave like a real session: buttons fire, bindings run, navigation works

 Roughly half of all "it doesn't work" is "you were still in Design mode." Flip to Preview constantly. Make it a twitch.

Perspective Property Editor (right) — the most important panel in the product:

Tab	Holds	It's really...
PROPS	<code>text</code> , <code>value</code> , <code>style</code> , <code>columns</code> , <code>data</code> — component-specific	<i>What this component is</i>
POSITION	How it sits in its parent container	<i>Where it goes</i> — and the meaning changes per container type
META	<code>name</code> , <code>visible</code> , <code>tooltip</code> , <code>domId</code>	<i>Identity and visibility</i>
CUSTOM	Properties you invent	<i>Your view's variables</i>

 **CUSTOM is not a scratchpad, it's your architecture.** Good Perspective views keep their state in custom properties on the root container, bind components to those, and never let components reach sideways into each other. This is [4.6](#) and it's the difference between a project you can still edit at screen fifty and one you can't.

Component Palette — search it, don't browse it. Faster every time.

Tag Browser — drag a tag onto the canvas and Ignition offers to make a bound component. Drag a tag onto a *property* and it makes a binding. That drag-to-bind gesture is the fastest thing in the Designer and beginners almost never use it.

Output Console (`Tools > Output Console`) — where `print` and script errors land while you're in the Designer. Keep it open. It is the difference between debugging and guessing.

2.3 Saving, and your safety net

`Ctrl+S` saves to the Gateway and open sessions update. `Ctrl+Z` / `Ctrl+Y` do what you expect.

8.3 stores project resources as **JSON files on disk**, which is what finally makes real source control practical (see [Part 11](#)). If your team has Git wired to the Gateway's project directory, your saves become reviewable commits.

If not: `File > Export` the view before a risky change. Ten seconds. I have never once regretted it and I have absolutely regretted skipping it.

⚠ On a production Gateway, **assume every save is live to operators**. There is no staging environment unless somebody built one. This is the single most important sentence in this Part and it has no jokes in it.

RECALL

1. Which Property Editor tab changes meaning depending on the parent container?
2. Where does `print` output go, and why not the browser console?
3. What's the ten-second insurance policy before a risky edit?

Answers: (1) POSITION. (2) the Gateway — Output Console in the Designer or the Gateway logs, because scripts run server-side. (3) File > Export the view.

Part 3. Views, and the Art of Not Drawing Pixels

 25 minutes · **The one thing:** a View is a reusable function with a UI, and the container type you pick decides how layout behaves forever.

3.1 The View is the atom

A **View** is a self-contained, reusable piece of interface. It has:

- a **root container**, chosen when you create it (and awkward to change later, so think for four seconds first)
- **params** — its inputs and outputs, exactly like function arguments
- **props** — view-level settings like `defaultSize`
- **custom** — view-scoped variables

Views get used three ways: as a **page's primary view**, as a **docked view** (nav bar, header, alarm banner), or **embedded inside another view**.

 **The Parameter Promise.** Design every view as if a stranger will embed it tomorrow. Take a parameter; don't reach for globals; never hardcode a tag path. Follow this one rule and your screens quietly become a component library. Break it and you get forty views that each work in exactly one place.

Folders. Pick a convention on day one, then don't renegotiate it at 4pm on a Friday:

```
Views/  
  Pages/           ← full screens a URL points at  
    Overview  
    Line1/Detail  
  Docks/           ← nav bars, headers, alarm banners  
    LeftNav  
    Header  
  Poupups/         ← popup and modal content  
    MotorFaceplate  
  Components/      ← small reusable pieces  
    ValueCard  
    StatusPill
```

3.2 Five containers, and the one you should default to

The container type decides what **POSITION** means for its children. New developers fight this harder than anything else in the product, and it's genuinely simple once somebody just says it:

Container	Children positioned by	Reach for it when
Coordinate	<code>x</code> , <code>y</code> , <code>width</code> , <code>height</code> — fixed px or percent	P&ID mimics, overlapping graphics, anything drawn to scale
Flex	<code>basis</code> , <code>grow</code> , <code>shrink</code> , <code>alignSelf</code>	Your default. Rows and columns that adapt
Column	Responsive grid that reflows by screen size	Dashboards of cards

Container	Children positioned by	Reach for it when
Breakpoint	Two children, swapped at a pixel threshold	Genuinely different phone vs desktop layouts
Tab	Children become tabs	Tabbed sections in one view

Coordinate has two modes. **Fixed** keeps children at pixel sizes. **Percent** scales them with the container — which is how a P&ID diagram grows onto a 4K control-room display without you rebuilding it.

Flex is the workhorse. Set `direction` to `row` or `column`, then per child:

- `basis` — starting size along the direction (`auto`, `100px`, `30%`)
- `grow` — share of *leftover* space this child absorbs; `0` means don't
- `shrink` — willingness to give up space when cramped
- `alignSelf` — cross-axis alignment

If you know CSS flexbox: it is CSS flexbox. If you don't, here's the only trick you need today — `grow: 0` on the header, `grow: 1` on the body, `grow: 0` on the footer. That's the classic header/body/footer layout and it's correct at every screen size ever made. It covers most screens you will build.

⚠ **The Coordinate Trap.** The number one layout mistake is building everything in a Coordinate container because it feels like drawing, then discovering on demo day that the screen is unusable on the tablet the plant manager brought. Default to **Flex**. Use Coordinate when you are drawing a picture of a physical thing, and for basically no other reason.

FIELD NOTE I've watched this exact demo go wrong twice. Both times the screen was beautiful on the developer's 27-inch monitor and unusable on the 10-inch panel it was destined for. Both times the rebuild cost more than the original build. Test on the real device early — not because it's good practice, but because the rebuild is genuinely expensive and entirely avoidable.

3.3 Parameters: in, out, both

On the View's `PARAMS` tab, each parameter is marked **input** (passed in), **output** (written back to the parent), or **in/out**. A faceplate might take `tagPath` in and hand `confirmed` back.

You pass params when you embed a view, open a popup, or navigate to a page whose URL carries parameters (`/line/:lineId`).

3.4 Embedding, and the component that makes you feel powerful

Component	Does	Use when
Embedded View	Renders one view inside another, passing <code>params</code>	Reuse a card, faceplate, header
Flex Repeater	Renders <i>N</i> copies of one view from an array of params	A list of pumps, a row of KPIs
View Canvas	Multiple views at explicit positions	Free-placement dashboards

Learn the Flex Repeater early. Its `props.instances` is an array where each element is one copy's parameters:

```
[
  { "tagPath": "[default]Line1/Pump1", "label": "Pump 1" },
  { "tagPath": "[default]Line1/Pump2", "label": "Pump 2" },
  { "tagPath": "[default]Line1/Pump3", "label": "Pump 3" }
]
```

Bind `instances` to a query or a script and your UI grows and shrinks with the plant by itself. This is the moment Perspective stops being "drawing screens" and starts being software, and it's worth pausing on when you hit it in [Lab 9](#).

△ Repeaters aren't free. Each instance is a full view with its own bindings. Twenty instances of a fifteen-binding view is three hundred live subscriptions — *per session*. It's a great feature and it will happily eat your Gateway if you stop thinking.

RECALL

1. Which container should you default to, and which is for drawing physical things?
2. What shape does `props.instances` want?
3. What's the Parameter Promise?

Answers: (1) Flex by default; Coordinate for pictures of physical things. (2) an array of objects, one per instance, each holding that copy's params. (3) design every view to take parameters and never hardcode a tag path, so it's embeddable anywhere.

Part 4. Bindings: Where the Magic Actually Is

 30 minutes · **The one thing:** bindings — not scripts — are how Perspective is meant to be built, and an indirect tag binding plus a view parameter is the trick behind every professional project you'll ever see.

This is the most important Part in the guide. If your attention is going to hold for exactly one Part today, spend it here.

4.1 The property tree is the whole API

Everything is a property on a JSON-ish tree: `props.text`, `props.style.backgroundColor`, `position.grow`, `meta.visible`, `custom.selectedMotor`. Bindings write to properties, scripts write to properties, the renderer reads properties. There's no hidden layer, no secret second system. That's genuinely all there is.

4.2 The seven binding types

Select a property, click the **chain-link icon** beside it, pick a type:

Binding	Reads	Notes
Tag	A tag's value or any tag property	Three flavors — below
Property	Another property in this view	The glue for local wiring
Expression	An Ignition expression	Cheap and fast
Expression Structure	Builds a JSON object/array from several expressions	Great for feeding charts and tables
Query	A Named Query (or inline SQL, but see 6.3)	Polled or triggered
Tag History	Historical data as a dataset	Feeds trends
HTTP	A REST endpoint	External web data

Tag bindings come in three flavors, and the second one is the one that matters:

1. **Direct** — you pick the tag. `[default]Line1/Tank1/Level`. Simple, static, fine.
2. **Indirect** — the path has placeholders filled from other properties:

```
[default]{1}/Level
```

with `{1}` bound to `view.params.equipmentPath`.

3. **Expression** — the whole path is computed. Most flexible, least readable.

 **The Faceplate Pattern.** Indirect tag binding + a view parameter = one view that serves every motor in the plant. That's it. That's the trick. Everything that looks impressive in a professional Ignition project is some variation of this one idea, and you can learn it in twenty minutes in [Lab 8](#).

Options worth knowing:

- **Bidirectional** (Tag and Property bindings) — writes flow back. A numeric input bound bidirectionally to a setpoint writes the setpoint as the operator types.
- **Overlay Opt-Out** — suppress the bad-quality overlay for this binding. Sparingly. See 1.4.
- **Enabled** — switch a binding off without deleting it. Underrated debugging tool.

△ An accidental bidirectional binding is how an operator changes a setpoint by *looking at a screen funny*. Tick that box on purpose or not at all.

4.3 Transforms, or how to stop making junk tags

A binding can carry a chain of **transforms** that reshape the value on the way to the property:

Transform	Does	Example
Map	Lookup table	<code>0 → "Stopped", 1 → "Running", 2 → "Fault"</code>
Format	Number/date formatting	<code>#,##0.0, yyyy-MM-dd HH:mm</code>
Expression	Expression where <code>value</code> is the input	<code>value * 2.20462</code>
Script	<code>def transform(self, value, quality, timestamp)</code>	Everything else

🔑 **The No-Junk-Tags Rule.** Need level as a percentage *and* as a bar color? One tag binding, two components, transforms doing the last mile. Do **not** create a `Level_Percent` memory tag for a display concern. Display concerns belong in the display. Junk tags are forever, and they breed.

△ **Script transforms are the most abused feature in Perspective.** They run on the Gateway every time the value changes, in every session. A `system.db.runQuery` inside a script transform on a one-second tag, open in forty sessions, is forty queries per second, and your DBA will find you. Try Map, then Expression, then — only if you must — Script, kept short and pure.

4.4 The expression language in ninety seconds

It is not Python. It's a small, fast, side-effect-free expression language.

```
// tag reference
{[default]Line1/Tank1/Level}

// property references
{view.params.lineId}
{../Label.props.text}           // relative path to a sibling

// conditionals
if({[default]Pump1/Running}, "RUNNING", "STOPPED")
case({[default]Pump1/State}, 0, "Off", 1, "Starting", 2, "Running", "Unknown")

// math & strings
```

```

round([[default]Tank1/Level] / [[default]Tank1/Capacity] * 100, 1)
concat("Line ", {view.params.lineId})

// null safety – you need this more often than you think
coalesce([[default]Maybe/Missing], 0)

// dates
dateFormat(now(), "yyyy-MM-dd HH:mm:ss")
dateDiff({session.props.startTime}, now(), "minute")

```

⚠ **now()** is a loaded gun. It takes a poll rate — `now(1000)` re-evaluates every second, and **forces the whole expression to re-evaluate at that rate**. `now(0)` evaluates once. A screen full of default-rate `now()` calls is a screen that never stops working, on a Gateway that never gets a break. Pass the argument deliberately, every time.

4.5 Where does this logic go? The table that settles arguments

Beginners scatter logic everywhere. Work down this list and stop at the first row that fits:

Need	Put it	Why
Show a tag	Tag binding	Cheapest thing available
Reformat or recolor for display	Transform on the binding	Stays with the display
Derived from several tags, one view needs it	Expression binding on a custom property	Local, visible, cheap
Derived from tags, many screens or alarms need it	Expression tag	Computed once on the Gateway, not once per session
Data from SQL	Named Query binding	Parameterized, cached, secured
Reusable Python	Project Library function	One copy, testable
Must run with nobody watching	Gateway Event	Sessions come and go

🔑 **The Nobody's-Looking Test.** *Does this need to be true when no one has the screen open?* Yes → it belongs in a tag or a Gateway event. No → it belongs in a binding in the view. This one question settles about 80% of "where should this go" arguments, including the ones you'll have with yourself at 11pm.

4.6 The view model pattern

This is the pattern that separates maintainable Perspective from the other kind:

1. On the **root container**, create `custom` properties describing the view's state — `custom.motorData`, `custom.selectedIndex`, `custom.isEditing`.
2. Bind those to tags, queries, expressions. **This is the only place data enters the view.**
3. Bind each component's display properties to those custom properties with simple **Property** bindings.
4. Component events write to custom properties — never to other components.

Why bother: one place to look when data is wrong, one place to change when the source changes, and components become swappable.

The alternative — a Label bound to a tag, a Gauge bound to the same tag, and a script reaching into `../../../../Flex/Label.props.text` to read it back out — works fine for about fifteen screens and then becomes a special kind of misery. I have been called in to rescue that project. Twice. The honest recommendation both times was "start over," which is a thing nobody wants to hear in a conference room.

RECALL

1. What two ingredients make the Faceplate Pattern?
2. A value is needed by six screens
and
an alarm. Binding or tag?
3. Why shouldn't you make a `Level_Percent` memory tag just to show a percentage?

Answers: (1) an indirect tag binding plus a view parameter. (2) an expression tag — it passes the Nobody's-Looking Test and gets computed once instead of per session. (3) it's a display concern; a transform on the binding does it with no junk tag to maintain forever.

Part 5. Scripts, Events, and Passing Notes

 25 minutes · **The one thing:** prefer configured actions to scripts, keep event scripts to three lines, and put real logic in the Project Library.

5.1 Most of the time, you don't need code

Select a component, open **Event Configuration** (the ⚡ icon, or right-click). Events are grouped: mouse (`onClick`, `onDoubleClick`, `onMouseEnter` ...), component-specific (`onActionPerformed` on a Button, `onChange` on inputs, `onRowClick` on a Table), keyboard, focus.

For each event, attach one or more **actions** — and look at what you get without writing Python:

Action	Does
Navigation	Go to a page, a view, or a URL
Popup	Open / close / toggle a popup, with params
Dock	Open / close / toggle a docked view
Set Property	Write a value to a property. No code.
Script	Run Python
Download File	Send a file to the browser

 **Configured over clever.** Prefer actions to scripts. They're visible in the Designer, they show up in search, and they don't hide behavior from whoever maintains this after you. A Set Property action is worth ten lines of Python you'll have to explain later.

5.2 Jython, and its four sharp edges

Ignition scripting is **Jython 2.7** — Python 2.7 on the JVM. Four things will bite you:

1. `1/2` is `0`. Integer division. Write `1.0/2`.
2. `print(x)` works; it goes to the Gateway, not a browser.
3. You can reach into Java classes when you need to, which is occasionally a lifesaver.
4. **No numpy, no pandas.** Third-party packages with C extensions don't work. Pure-Python libraries can be added, but plan to lean on SQL and Ignition's own functions instead.

⚠ That fourth one catches data people hardest. If your instinct is "I'll just pandas this," stop and write a Named Query. The database is right there and it's better at this than you are.

Inside a Perspective event script:

```
def runAction(self, event):
    # self                -> the component this is attached to
    # self.view           -> the view
    # self.view.params.tagPath -> a view parameter
    # self.getSibling("Label") -> a sibling component
    # self.parent         -> the parent
    # self.session        -> session object (props, user info)
    # self.page           -> page object
    pass
```

Writing properties and tags:

```
def runAction(self, event):
    self.getSibling("StatusLabel").props.text = "Working..."
    self.view.custom.busy = True

    qv = system.tag.readBlocking(["[default]Line1/Pump1/Speed"])[0]
    speed = qv.value

    system.tag.writeBlocking(["[default]Line1/Pump1/SpeedSetpoint"], [1750])
```

△ Notice those functions take **lists** and return **lists**. That's a hint, and it's not subtle. `readBlocking` and `writeBlocking` block a Gateway thread — never loop over five hundred tags one at a time when you could pass five hundred paths in a single call. Same work for you, a fraction of the cost for the server.

5.3 The Project Library, and the three-line rule

Project Browser > Scripting > Project Library. Make a package (`plant`), a module (`motors`):

```
# Project Library > plant > motors
def start(tagPath):
    """Command a motor to start. Returns True if the write was accepted."""
    result = system.tag.writeBlocking([tagPath + "/StartCmd"], [True])
    return result[0].good

def summarize(tagPaths):
    """Read Running/Fault for many motors in ONE call."""
    paths = []
    for p in tagPaths:
        paths.append(p + "/Running")
        paths.append(p + "/Fault")
    values = system.tag.readBlocking(paths)
    out = []
    for i, p in enumerate(tagPaths):
        out.append({
            "path": p,
            "running": values[i * 2].value,
            "fault": values[i * 2 + 1].value,
        })
    return out
```

Then from anywhere:

```
def runAction(self, event):
    plant.motors.start(self.view.params.tagPath)
```

 **The Three-Line Rule.** Event scripts get three lines: validate, call the library, handle the result. All real logic lives in the Project Library, where it's reusable, greppable, and testable. I enforce this on every team I've run and I've never regretted it once.

Tools > Script Console runs Python against the Gateway right now. It's your REPL. Develop every non-trivial function there first, where the feedback loop is two seconds instead of save-preview-click-squint.

5.4 Passing notes: message handlers

The nav bar needs to tell the main view "refresh." They're different views and can't see each other. So they pass notes.

Sender:

```
def runAction(self, event):
    system.perspective.sendMessage(
        "refreshData",
        payload={"lineId": 1},
        scope="page"           # "page", "session", or "view"
    )
```

Receiver — on the target component or view, Event Configuration > **Message Handlers** > add `refreshData` with a matching scope:

```
def onMessageReceived(self, payload):
    self.view.custom.lineId = payload["lineId"]
    self.refreshBinding("custom.data")
```

Scope	Reaches
view	Handlers in the same view instance
page	Every view on this page, docks and popups included
session	Every page in this user's session

 **Broadcast, don't reach.** A popup that saves a record shouldn't know who opened it. It announces "recordSaved" to the room and whoever cares reacts. This is how you keep views from growing tentacles into each other.

△ Ninety percent of "my message handler doesn't fire" is a scope mismatch — sent `page`, listening on `view`. The other ten percent is testing in single-view preview, where the dock you're sending from doesn't exist.

5.5 Things that run without anyone watching

Project Browser > Scripting:

Gateway Events — no session required: **Timer** (every N seconds), **Tag Change**, **Startup/Shutdown**, **Message** handlers.

Session Events — per Perspective session: **Startup/Shutdown**, **Page Startup**, **Authentication** (a great place to set `session.custom` defaults once a user logs in).

△ Gateway timer scripts are where performance goes to die. Every one you add runs forever on production, whether or not it's still needed, long after you've moved on. Before writing one, ask whether a tag change event would do the same job with less blast radius. Usually it would.

RECALL

1. How many lines should an event script have, and where does the real logic go?
2. Your message handler isn't firing. What's the first thing to check?
3. Why do `readBlocking` and `writeBlocking` take lists?

Answers: (1) three — validate, call the Project Library, handle the result. (2) scope mismatch between sender and handler. (3) so you can read or write many tags in one Gateway call instead of one call per tag.

Part 6. Tags, History, and Not Getting Fired by SQL



25 minutes · **The one thing:** Named Queries, always — and history you didn't enable yesterday doesn't exist today.

6.1 Getting real data in

In the Gateway web UI:

- **OPC UA** — Ignition ships an OPC UA server and drivers (Allen-Bradley, Siemens, Modbus TCP...). Make a **Device Connection**, browse it in the Designer's OPC Browser, drag addresses into your tag tree.
- **Database Connections** — JDBC to SQL Server, Postgres, MySQL. The connection has a name; your queries reference that name.
- **MQTT** — via Cirrus Link modules, if installed.

For training you don't need a PLC. Memory tags plus expression tags simulate a plant perfectly well, and Ignition ships device simulators you can add as a connection to get moving values for free. [Lab 2](#) uses memory tags; [Lab 12](#) suggests the simulator.

6.2 History

Enable history per tag — or better, on the **UDT definition**, once, for every instance forever:

- **Sample mode / scanclass** — how often to consider storing
- **Deadband** — don't store unless it moved by X. This is what keeps your database from becoming a landfill.
- **Max time between samples** — store periodically even when nothing changes, so a gap means "outage" rather than "who knows"

Then bind a chart to a **Tag History binding**: pick tag paths, a time range (or bind the range to date pickers), an aggregation mode (`Average`, `MinMax`, `LastValue` ...).

Chart components: **Power Chart** (operator-facing — users pick their own pens, and you get an enormous amount of functionality for one drag), **Time Series Chart** (you choose the pens), **XY Chart**, tables.

⚠ **History is not retroactive.** It starts the moment you enable it. Every single person learns this by needing yesterday's data on a tag they set up this morning. Enable history on anything remotely interesting *early* — storage is cheap and time machines aren't for sale.

FIELD NOTE Everybody learns this the same way. Somebody walks over and asks why the oven ran cold last Thursday, and you discover the tag has been faithfully reporting its value to nobody since commissioning. There is no recovering it. The tag was right there the whole time. Storage is cheap; a time machine is not for sale at any price. Historize anything remotely interesting on the day you create it.

8.3 restructured the historian — there's now a Historian Core Module, a SQL Historian Module, and a public Historian API, with a fast built-in time-series store. Binding to history in the Designer feels the same; what changed is Gateway-side configuration. When you set history up, check [Config > Historian](#) and note which provider you're writing to.

6.3 Named Queries, or how not to get fired

[Project Browser > Named Queries](#). A Named Query is stored, parameterized, permission-checked, optionally cached SQL. Types: **Query** (a dataset), **Update Query** (INSERT/UPDATE/DELETE), **Scalar Query** (one value).

```
-- Named Query: Production/GetShiftTotals
SELECT line_id, SUM(units) AS total
FROM production_log
WHERE ts >= :startDate AND ts < :endDate
GROUP BY line_id
ORDER BY line_id
```

Parameters get declared and typed on the Authoring tab. Bind a component's `props.data` with a **Query** binding, then bind each parameter to a date picker.

	Named Query	String SQL in a script
SQL injection	Parameters are bound — safe	You are one <code>+</code> from an incident
Caching	Built in	None
Security	Per-query permissions	None
Reuse	One definition	Copy-paste forever
Finding it in a year	It's in the tree	Buried in a script somewhere

 **No string SQL. Ever.** Not "just for testing." Not "just this once." The test version is what ships; it always is. This is the one rule in this guide I'd call non-negotiable, because the failure mode isn't a bug, it's a breach.

 **Value vs Literal parameters.** `Value` parameters are properly bound and safe. `Literal` parameters are string-substituted into the SQL — necessary for dynamic table or column names, and exactly as dangerous as that sounds. Never wire a Literal parameter to anything a user can type into.

From script when you need it:

```
data = system.db.runNamedQuery("Production/GetShiftTotals",
                               {"startDate": start, "endDate": end})
```

6.4 Datasets vs arrays: the afternoon-eater

Two shapes of tabular data, and confusing them costs everyone one afternoon exactly once:

- **Dataset** — Ignition's native table type. What SQL and tag history hand back.
- **JSON array of objects** — what Perspective components generally want in `props.data`.

Query bindings usually offer a return format that gives the component what it wants. In script:

```
# dataset -> list of dicts
data = system.dataset.toPyDataSet(ds)
rows = [{"line": row["line_id"], "total": row["total"]} for row in data]
```

△ **Table shows nothing, threw no error?** It's this. It's essentially always this. Check the shape before you check anything else.

RECALL

1. Why Named Queries instead of building SQL strings? Give the reason that isn't convenience.
2. You need last month's trend for a tag you historized this morning. What do you have?
3. Your Table is blank and there's no error in the logs. First suspect?

Answers: (1) bound parameters make SQL injection impossible; string concatenation makes it inevitable. (2) nothing — history isn't retroactive. (3) data shape — a dataset where the component wants an array of objects.

Part 7. Pages, Docks, and Popups

 20 minutes · **The one thing:** map URLs to views in Page Configuration, set your docks in Shared Settings once, and navigate by page so the back button works.

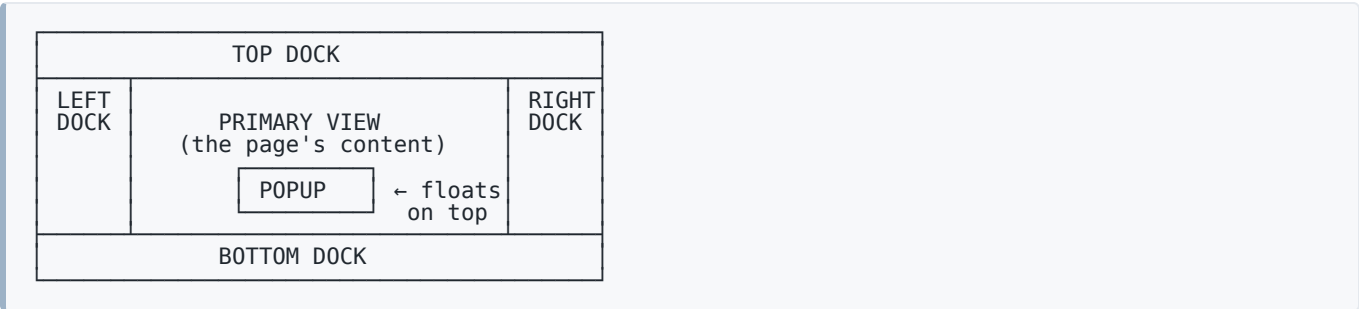
7.1 URLs are a feature, use them

Project Browser > Perspective > Page Configuration maps **URL paths** → **views**:

URL	Primary View
/	Pages/Overview
/line/:lineId	Pages/LineDetail
/reports	Pages/Reports

A `:parameter` segment becomes a view parameter — `/line/3` passes `lineId = "3"` in. Which means your screens are **linkable**: an operator can bookmark one machine, and you can email someone a URL that opens exactly the screen you're arguing about. Almost nobody uses this. You should.

7.2 The regions of a page



Docked views are configured in Page Configuration. Each dock gets an **ID** — name it, because you'll need it for `system.perspective.toggleDock("leftNav")`. Settings that matter:

- **Display** — `visible` (always), `onDemand` (opens when asked), `auto` (breakpoint-driven)
- **Push vs overlay** — does the dock shove content aside or float over it
- **Size, resizable, modal, auto-dismiss**
- **Show when** — breakpoint conditions, so the same nav is a permanent sidebar on a desktop and a hamburger drawer on a phone

 **Shared Settings, once.** Configure your nav and header in Page Configuration's Shared Settings so every page inherits them, then override for the odd full-screen kiosk view. Do not paste your nav bar into nine pages. You will find the tenth one three months later, still showing last quarter's menu.

7.3 Popups

```
system.perspective.openPopup(  
  id="motorFaceplate",           # unique handle – reopening this id reuses it  
  view="Popups/MotorFaceplate",  
  params={"tagPath": "[default]Line1/Pump1"},  
  title="Pump 1",  
  modal=True,  
  draggable=True,  
  resizable=False,  
  showCloseIcon=True  
)  
system.perspective.closePopup("motorFaceplate")
```

The **Popup action** on an event does all this without code, so prefer that.

⚠ The **id** is your only handle on a popup. Generate ids dynamically and you can't close them programmatically — they just stack up on the operator like browser tabs, until someone files a ticket titled "screens everywhere."

7.4 Navigation

```
system.perspective.navigate(page="/line/3")           # internal page  
system.perspective.navigate(url="https://example.com") # external  
system.perspective.navigate(view="Pages/Detail", params={...}) # swap the primary view
```

🔑 **Navigate by page, not by view.** Page navigation updates the URL, works with the browser back button, and is bookmarkable. View navigation does none of that, and operators press Back anyway — then call you.

7.5 Session and page properties

```
session.props.auth.user.userName    # who's logged in  
session.props.auth.user.roles  
session.props.auth.authenticated  
session.props.device.type           # desktop / mobile / tablet  
session.props.address               # client IP  
session.props.theme  
session.props.locale  
  
session.custom.*                    # YOUR session-scoped variables  
page.props.pageId                   # unique id for this page instance
```

`session.custom` is the right home for "the line this operator picked" or "their preferred units." Set it in a Session Startup or Authentication event, read it anywhere. It's per session, so two operators can be looking at different lines in the same project at the same time — which is exactly what you want and surprisingly easy to get wrong by reaching for a memory tag instead.

RECALL

1. Page navigation or view navigation, and why?

2. Where do you configure a nav bar so all pages get it?
3. What breaks if you give popups dynamically generated ids?

Answers: (1) page — it updates the URL, so bookmarks and the back button work. (2) Page Configuration > Shared Settings. (3) you can't close them from script, so they pile up.

Part 8. Style Without Regret

 15 minutes · **The one thing:** put colors in style classes, bind `props.style.classes`, and never name a class after a color.

8.1 Three levels, in order of preference

1. **Inline style** — `props.style.backgroundColor` on one component. Fine once. Terrible as a habit.
2. **Style Classes** — Perspective > Styles. Define `status.running`, `status.fault`, `card.base` once; apply with `props.style.classes`. **This is where your styling belongs.**
3. **Themes** — Gateway-level palettes (`light`, `dark`, custom). Switch at runtime with `system.perspective.setTheme("dark")`.

 **The Bindable Class List.** `props.style.classes` takes a **space-separated list** and it can be **bound**. That's the whole trick for state-driven styling:

```
// expression binding on props.style.classes
"card.base " + case({[default]Pump1/State},
  0, "status.stopped",
  1, "status.starting",
  2, "status.running",
  "status.unknown")
```

One binding, no scripts, colors defined in exactly one place. When the plant standard goes from green to teal, you edit one style class instead of four hundred components.

8.2 Name for meaning, not appearance

Call it `status.fault`, not `text.red`.

In two years the fault color will change, and a class named `text.red` that renders blue is a special kind of misery — the kind where the code is technically correct and everyone reading it is quietly furious.

Build a small system before your first real screen:

```
Styles/
status/   running, stopped, fault, warning, disabled
text/     title, subtitle, body, caption, value.large
card/     base, header, body
layout/   page.padding, section.gap
```

8.3 Symbols, and the 8.3 Drawing Editor

Perspective ships a **Symbol** library — motors, valves, pumps, vessels — with state properties built in. Bind `props.value` to a state tag and the symbol animates. Use these before you draw anything yourself.

New in 8.3: the Drawing Editor. A real vector illustration tool inside the Designer — paths, fills and strokes, snapping, guides, layering, animation. Custom P&ID elements without a round trip through Illustrator and an SVG import. It's a genuine upgrade and also a magnificent way to lose a Thursday, so finish the labs first.

RECALL

1. What's wrong with a style class named `text.red`?
2. What property do you bind for state-driven coloring, and what does it accept?

Answers: (1) it names an appearance, not a meaning — when the standard changes, the name lies. (2) `props.style.classes`, which takes a bindable space-separated list of class names.

Part 9. Security That Means Something



15 minutes · **The one thing:** hiding a button is not security — enforce on the tag and the query, then hide the button too.

9.1 The pieces

- **Identity Providers (IdP)** — where users come from: Ignition's internal source, LDAP/AD, or OIDC/SAML SSO.
- **Users and Roles** — the classic model.
- **Security Levels** — 8.x's hierarchical model, a tree like `Authenticated/Roles/Operator`. More expressive than flat roles.
- **Security Zones** — *where* the request comes from (IP ranges, hosts). "Setpoint changes only from the control-room subnet" is a zone rule.
- **Permissions** — combine them: *Supervisor and ControlRoom zone*.

9.2 Applying it

On components — component permissions let you hide or disable when the user lacks permission. Put this on the Button that starts the pump, not just on the page holding it.

In expressions and scripts:

```
// expression binding on meta.visible
isAuthorized(false, "Authenticated/Roles/Supervisor")
```

```
if system.perspective.isAuthorized(
    isAllOf=False,
    securityLevels=[{"name": "Authenticated/Roles/Supervisor"}]):
    ...
```

On Named Queries — per-query permissions, so the *data* is protected, not just the button that asks for it.



The Hidden Button Fallacy. Hiding a control stops a confused operator. It does not stop a determined one, and it does nothing at all about anything that isn't your UI. Real enforcement lives on the tag's write permissions, on the Named Query, and in the security zone. Do both — hide it *and* enforce it — because the hidden button is good UX and the enforcement is the actual security.

⚠ Testing your permissions while logged in as an admin proves nothing. Make two throwaway users, `op1` and `sup1`, and actually log in as both. [Lab 18](#) walks it.

9.3 Auditing

Turn on an **Audit Profile** and point your project at it. Tag writes, query executions and logins get logged with who, what, and when.

On anything touching production, turn this on day one. The first time somebody asks who changed the setpoint at 2am, you will either have an answer or a very long meeting. You cannot retroactively audit — the past is not instrumented.

RECALL

1. Name the fallacy, and the complete fix.
2. What does a security

zone

check that a role doesn't?

Answers: (1) the Hidden Button Fallacy — enforce on the tag's write permissions and the Named Query, and hide the control as well. (2) where the request comes from — IP range or host.

Part 10. Alarms



15 minutes · **The one thing:** alarms are configured on the *tag* (ideally the UDT definition), and Display Path plus Associated Data are what make them useful at 3am.

10.1 Configuring

Alarms live **on the tag**, not on the screen. Select a tag, open the Alarms editor, add one:

- **Name, Priority** (Diagnostic → Critical)
- **Mode:** Above Setpoint, Below Setpoint, Between, Equal, Bit State, On Change...
- **Setpoint, Deadband, Time On/Off Delay** — delays are how you stop an alarm from chattering a thousand times at the threshold
- **Display Path** — the operator-friendly name
- **Associated Data** — extra context captured at alarm time: batch number, operator, line speed



Configure alarms on the UDT definition. Every instance inherits them; override per instance where you must. Configure them on instances instead and adding one alarm becomes a three-hour data-entry job, which is how you learn this rule the expensive way.

⚠ **Set the Display Path. Always.** An alarm table full of raw tag paths is hostile to the person reading it during an actual event. And fill in **Associated Data** — it's the thing everybody skips and then desperately wishes they had during the post-incident review. Capturing the batch number takes eight seconds now and saves an hour of forensics later.

10.2 Displaying

- **Alarm Status Table** — live alarms; acknowledge and shelve from it
- **Alarm Journal Table** — history of alarm events, from the journal database

Both filter by source path, priority, state, display path. A common pattern: a permanent bottom dock with a compact filtered Alarm Status Table as a banner.

10.3 Notification

The Alarm Notification module plus pipelines handles email, SMS, voice and escalation rosters. Out of scope for week one — but know that *"an alarm happened"* and *"a human was told"* are two separate systems, and only one of them is on by default.

RECALL

1. Where do you configure an alarm so 200 instances get it?
2. What two alarm fields do beginners skip and later regret?

Answers: (1) on the UDT definition. (2) Display Path and Associated Data.

Part 11. What's New in 8.3



10 minutes · **The one thing:** most tutorials you'll find online were written for 8.1 — this list is why your screen looks different.

You're learning on 8.3.9, so you get all of this free. But the internet is still mostly 8.1, and when a video's menu path doesn't exist on your Gateway, the reason is usually right here.

Perspective

- **Drawing Editor** — native vector illustration in the Designer: paths, fills and strokes, snapping, guides, layering, animation.
- **Form generator** — configure sections, fields, conditional logic and validation rules once; the component builds the responsive layout, the data logic, and client-side validation.
- **Offline mode** — the mobile app can take form entry with no connectivity and sync when the link comes back. Genuinely new capability for remote sites.

Gateway

- **Redesigned web interface** with global search. If an older tutorial says "go to Configure > something" and that path is gone, search for the page name instead of hunting menus.

Historian

- Restructured into a **Historian Core Module**, a **SQL Historian Module**, and a public **Historian API**, with a fast built-in time-series store.

Event Streams (new resource type)

- A structured pipeline: a **source** (tag change, Gateway event, HTTP, Kafka, database) → **encoding / filtering / transformation** stages → a **handler** (script, database table), with a dedicated **error-handling stage** and a **test mode**. This is the clean answer to a whole category of problems people used to solve with a pile of Gateway tag-change scripts. You won't need it in week one; know the name so you recognize it when you do.

Project storage

- Project resources are stored as **JSON files on disk**, making change tracking, merging and Git workflows actually practical. This is 8.3's big structural change and the reason a lot of shops upgraded.

△ Point releases (8.3.1 → 8.3.9) move things. When a behavior surprises you, skim the release notes between the tutorial's version and yours before assuming you're wrong.

RECALL

1. Why does an 8.1 tutorial's Gateway menu path sometimes not exist for you?
2. What 8.3 resource replaces a pile of Gateway tag-change scripts?

Answers: (1) 8.3 redesigned the Gateway web UI — use global search. (2) Event Streams.

Part 12. The Seven-Day Plan

Seven sessions of two to three hours, then a capstone. Each session: read the Part, build the labs, then do the 🧠 Recall boxes without looking.

Short on time? Do **Days 1, 2 and 4**. Those three cover about 70% of daily Perspective work, and Day 4 contains the idea that makes you look like you know what you're doing.

Day 1 — First live value 🕒 2–3 h

Read Parts 1 and 2. **Build** [Lab 1](#), [Lab 2](#), [Lab 3](#).

🚩 **Boss fight:** a number changes in your browser because you changed a tag in the Designer — and you can say out loud which machine did the computing.

Day 2 — Layout that survives a tablet 🕒 2–3 h

Read Part 3. **Build** [Lab 4](#), [Lab 5](#), [Lab 6](#).

🚩 **Boss fight:** drag your browser from desktop width down to phone width. The screen stays usable and never grows a horizontal scrollbar.

Day 3 — Bindings and transforms 🕒 2–3 h

Read Part 4 (the big one). **Build** [Lab 7](#), [Lab 8](#).

🚩 **Boss fight:** one view displays any tank in the plant based on a parameter, and three copies of it sit on a screen pointed at three different tanks.

Day 4 — Reuse: the day it clicks 🕒 2–3 h

Read 3.3–3.4 again, and 4.6. **Build** [Lab 9](#), [Lab 10](#).

🚩 **Boss fight:** you add a pump to the tag tree and a card appears on your screen **without you opening the view**. If you only get one day of this plan, get this one.

Day 5 — SQL, history, charts 🕒 2–3 h

Read Part 6. **Build** [Lab 11](#), [Lab 12](#).



Boss fight: a table re-queries when you change a date picker, and a trend shows a tag's real recorded history.

Day 6 — Scripting and events 🕒 2–3 h

Read Part 5. **Build** [Lab 13](#), [Lab 14](#), [Lab 15](#).



Boss fight: a button in the nav dock refreshes a view in the page center that it has no reference to.

Day 7 — Alarms, security, polish 🕒 2–3 h

Read Parts 8, 9, 10, 15. **Build** [Lab 16](#), [Lab 17](#), [Lab 18](#).



Boss fight: you caused an alarm and acked it, your colors come from style classes, and a button is hidden *and* enforced for the wrong user.

The Capstone: Line Monitoring Application



4+ hours · No steps this time. Build it.

1. A **Motor** UDT: **Running**, **Fault**, **Speed**, **SpeedSetpoint**, **RunHours**. History on **Speed** and **RunHours**. A high-speed alarm and a fault alarm **on the definition**.
2. At least six instances across two lines.
3. A left nav dock (line selection) and a top header dock (plant name, clock, logged-in user, alarm count).
4. An **Overview** page: a Flex Repeater of motor cards, driven by a query or a browse script — not a hardcoded list.
5. Clicking a card opens a **faceplate popup** for that motor: live values, a writable speed setpoint, start/stop, a one-hour trend.
6. Start/stop is **hidden and enforced** for non-Supervisors.
7. All colors from style classes. Works in light and dark.
8. A **Reports** page: date range + table from a Named Query.
9. Usable at 1920px and at 400px.

Then grade yourself honestly:

- Is there a single hardcoded tag path in any reusable view? (There shouldn't be.)
- Does any script outside the Project Library exceed ten lines?
- Does any component reach into another component by relative path?
- How many bindings are on your heaviest screen? Could any be an expression tag instead?
- Open it on your actual phone. Be honest. Is it usable?

That's roughly the shape of a real first assignment. Build it and you're employable on Perspective — not "familiar with," *employable on*.

Part 13. The Labs

Each lab: **Goal** → **Steps** →  **Boss fight** → **How this bites you**.

That last section is symptom-first on purpose. Recognizing a symptom is what actually saves you later; knowing a rule in the abstract mostly doesn't.

Lab 1: Get Oriented

 20 min · Pain: 

Goal: know where things live.

1. Browse to your Gateway (`http://<host>:8088`) and log in.
2. Find **Status** (health, sessions, performance) and **Config** (connections, security, historian). In 8.3's redesigned UI, use **global search** — much faster than hunting menus.
3. Open `Status > Perspective Sessions` . You'll come back here constantly.
4. Install the **Designer Launcher**, add your Gateway, launch the Designer.
5. Create a project named `training` , no parent, Perspective enabled.
6. Open `Tools > Script Console` and run:

```
print(system.date.now())
print(system.tag.readBlocking(["[System]Gateway/SystemName"])[0].value)
```

 The console prints your Gateway's name. You just ran code on a server from your desk — that's the entire architecture demonstrated in one line.

How this bites you: you installed the Vision Client Launcher (different download, same page). Or you used `localhost` from a machine that isn't the Gateway.

Lab 2: Make Some Tags

 25 min · Pain: 

Goal: a simulated tank, so you have data without a PLC.

1. In the **Tag Browser**, right-click `Tags` (the `default` provider) → `New Tag > Folder` . Name it `Line1` , then a subfolder `Tank1` .
2. Inside `Tank1` , create:

Name	Type	Data Type	Value / Expression
Capacity	Memory	Float	1000
Level	Memory	Float	450
Setpoint	Memory	Float	500
PumpRunning	Memory	Boolean	false
LevelPercent	Expression	Float	<code>{{[.]Level}} / {[.]Capacity} * 100</code>
Status	Expression	String	<code>if({[.]LevelPercent} > 90, "HIGH", if({[.]LevelPercent} < 10, "LOW", "NORMAL"))</code>

That `{{[.]Level}}` means "relative to this tag's folder." Use it — it makes folders copy-pasteable, which you'll exploit in Lab 8.

3. Change `Level` to `950` and watch `LevelPercent` and `Status` recompute instantly.

 Editing one tag changes two others with no code anywhere in the system.

How this bites you: your percentages come out truncated (you made the expression tag an Integer). Or nothing computes because you typed the expression into the *value* field instead of the **Expression** field.

Lab 3: Hello, Tag

 30 min · Pain: 

Goal: first view, first binding, first session.

1. **Perspective > Views** → right-click → **New View**. Name `Pages/Overview`, root container **Flex**. If offered, set it as the primary view for a new page at URL `/`.
2. Drag a **Label** onto the canvas.
3. Select it, find `props.text`, click the **chain-link** binding icon.
4. Choose **Tag**, browse to `[default]Line1/Tank1/LevelPercent`, OK.
5. Add a **Format** transform: `#,##0.0`.
6. Drag a **Cylindrical Tank** (or Simple Gauge). Bind `props.value` to the same tag.
7. Drag a **Numeric Entry Field**. Bind `props.value` to `[default]Line1/Tank1/Setpoint` and check **Bidirectional**.
8. `Ctrl+S`.
9. Toggle **Preview Mode**. Type a new setpoint. Check the Tag Browser — the tag changed.
10. Open the real session from the Gateway home page (or `http://<host>:8088/data/perspective/client/training`). Put it beside the Designer, change `Level` in the Tag Browser, watch the browser update.

 Two windows stay in sync with no refresh, and you understand why.

How this bites you: you forgot to save, so the session shows the last saved state. You bound `props.text` on the tank instead of `props.value`. You expected the bidirectional write to happen in Design mode — it needs Preview or a real session.

Lab 4: Flex Layout

 25 min · Pain: 

Goal: the header/body/footer layout that underlies most screens.

1. New view `Sandbox/FlexPractice`, root **Flex**.
2. Select the root, set `props.direction = column`.
3. Drop three **Flex Containers** in. Rename them (META > name) `Header`, `Body`, `Footer`.
4. `Header`: `position.basis = 60px`, `grow = 0`, `shrink = 0`.
5. `Footer`: `basis = 40px`, `grow = 0`, `shrink = 0`.
6. `Body`: `grow = 1`, `basis = auto`.
7. Give each a different background color temporarily so you can see what's happening.
8. Select `Body`, set `props.direction = row`, drop three Labels in, set each to `position.grow = 1`. They split the width evenly.
9. Change one Label's `grow` to `2`. It takes half. **That's the entire flex model.**
10. Preview and resize. Header and footer hold; body absorbs.

 You can predict what a `grow` change will do before you make it.

How this bites you: you set `grow` on the root instead of its children. Or you left `basis` at a fixed pixel value and `grow` looks broken — it only distributes *leftover* space, and you left none.

Lab 5: Coordinate and Percent Mode

 20 min · Pain: 

Goal: know when the other tool is right.

1. New view `Sandbox/CoordPractice`, root **Coordinate**.
2. Leave `props.mode` at `fixed`.
3. Drop a **Rectangle** shaped like a tank, then a **Cylindrical Tank** overlapping it. Notice overlap is trivial here and impossible in Flex. That's the whole point of this container.
4. Add a Label positioned over the tank, bound to `[default]Line1/Tank1/Status`.
5. Preview and resize — everything stays pixel-fixed.
6. Set root `props.mode = percent`. Resize again — everything scales proportionally.
7. Ask yourself: P&ID mimic on a 4K control-room display? (Percent.) A data entry form? (Neither — Flex.)



You can state the rule cold: *Coordinate for pictures of physical things, Flex for everything else.*

Lab 6: A Real Page with Docks



35 min · Pain: 🌶️🌶️

Goal: a shell you'll reuse for the rest of your career.

1. Create `Views/Docks/LeftNav`, root **Flex**, `direction = column`. Add three **Buttons**: "Overview", "Line 1", "Reports".
2. Create `Views/Docks/Header`, root **Flex**, `direction = row`. Add:
 - a Label with the plant name (`position.grow = 0`)
 - an empty Flex container with `position.grow = 1` (a spacer — this is the idiom)
 - a Label, expression-bound to `now(1000)`, Format transform `HH:mm:ss`
 - a Label, expression-bound to `{session.props.auth.user.userName}`
3. Open **Perspective > Page Configuration**.
4. Select **Shared Settings**. Add a **Left** dock: view `Docks/LeftNav`, **ID** `leftNav`, size `220`, display `visible`.
5. Add a **Top** dock: `Docks/Header`, **ID** `header`, size `60`, display `visible`.
6. Confirm `/` maps to `Pages/Overview`. Add a page `/reports` with a stub view.
7. On the "Overview" button: Event Configuration → `onActionPerformed` → **Navigation** action → Page → `/`. Repeat for the others.
8. Save, open a session, click around. Watch the URL change. **Press the browser back button.** It works, because you used page navigation.



A persistent nav and header on every page, configured in exactly one place.

How this bites you: you configured the docks on one page instead of Shared Settings, and now you're copying them. Or you skipped the dock ID and can't call `toggleDock` later.

Lab 7: Five Bindings



30 min · Pain: 🌶️🌶️

Goal: fluency with the binding dialog. Build `Sandbox/BindingZoo` with five Labels.

1. **Tag binding** — `props.text` → `[default]Line1/Tank1/Level`.
2. **Expression binding:**

```
"Tank is " + toString(round([default]Line1/Tank1/LevelPercent, 1)) + "% full"
```


3. **Property binding** — on the root, CUSTOM tab, add `custom.tankName = "Tank 1"`. Bind Label 3's `props.text` to `view.custom.tankName`. Change the custom property; watch the label follow.
4. **Tag binding + Map transform** — bind to `[default]Line1/Tank1/PumpRunning`, add a **Map** transform: `true → "Pump Running"`, `false → "Pump Stopped"`.
5. **Expression binding on `props.style.classes`** — after Lab 17, or use an inline color now.

Then bind `meta.visible` on Label 4 to `{[default]Line1/Tank1/PumpRunning}` and toggle the tag. Watch it appear and vanish. `meta.visible` being bindable is one of the most useful facts in the product and it's easy to go months without noticing.



You stop hunting for the binding icon.

Lab 8: The Faceplate Pattern



40 min · Pain: 🌶️🌶️🌶️ · **The most important lab here.**

Everything professional in Perspective is a variation on this. If you do one lab, do this one.

Goal: one view that displays *any* tank, chosen by a parameter.

1. Make a second tank: in the Tag Browser, copy `Line1/Tank1`, paste as `Line1/Tank2`. Change `Tank2`'s `Level`.
2. Create `Views/Components/TankCard`, root **Flex**, `direction = column`.
3. Select the **View** — the top node in the Project Browser for this view, *not* the root container. Go to **PARAMS**. Add: name `tankPath`, direction **input**, value `[default]Line1/Tank1` (a design-time default).
4. Add a title Label. Bind `props.text` by **Expression**:

```
{view.params.tankPath}
```

Ugly, but seeing the path prove itself is the point right now.

5. Add a level Label. Bind `props.text` with a **Tag binding**, switch the mode to **Indirect**, set the path to:

```
{1}/LevelPercent
```

then bind reference `{1}` to `view.params.tankPath`.

6. Add a Cylindrical Tank → `props.value` → indirect `{1}/LevelPercent`, same reference.
7. Add a status Label → indirect `{1}/Status`.
8. Save.
9. Back in `Pages/Overview`, drag an **Embedded View**:
 - `props.path` = `Components/TankCard`
 - `props.params` = `{ "tankPath": "[default]Line1/Tank1" }`
10. Duplicate it. Change the copy's param to `[default]Line1/Tank2`.
11. Preview. Two independent tank cards from **one** view definition.
12. Now change something in `TankCard` — a color, a font size. Save. **Both** cards change.



You understand that you just built a *component*, not a screen. Sit with that for a second; it's the hinge the whole product turns on.

How this bites you: you put the parameter on the root container's CUSTOM tab instead of the View's PARAMS tab — different nodes, click the very top item. You left a trailing slash and the path became `[default]Line1/Tank1//LevelPercent`. You expected the design-time default to apply at runtime; it doesn't.

Lab 9: Flex Repeater



35 min · Pain: 🌶️🌶️🌶️

Goal: stop hardcoding the list.

1. On `Pages/Overview`, delete the two Embedded Views from Lab 8.
2. Drop a **Flex Repeater**. Set `props.path = Components/TankCard`.
3. Set `props.direction = row`, `props.wrap = wrap` so cards flow.
4. Edit `props.instances`:

```
[
  { "tankPath": "[default]Line1/Tank1" },
  { "tankPath": "[default]Line1/Tank2" }
]
```

5. Preview. Two cards.
6. **Now make it dynamic.** Project Library:

```
# Project Library > plant > tanks
def listInstances(folderPath):
    """Return Flex Repeater instances for every tank folder under folderPath."""
    results = system.tag.browse(folderPath, {"tagType": "Folder"})
    instances = []
    for tag in results.getResults():
        instances.append({"tankPath": str(tag["fullPath"])})
    return instances
```

7. Bind `props.instances` with an **Expression** binding of `now(10000)` — a deliberate, slow heartbeat — then add a **Script** transform:

```
def transform(self, value, quality, timestamp):
    return plant.tanks.listInstances("[default]Line1")
```

8. Save. Add `Line1/Tank3` in the Tag Browser (copy/paste Tank1 again).
9. Within ten seconds a third card appears. **You did not touch the view.**



Adding a tank to the tag tree adds a card to the screen. This is the Day 4 boss fight and the thing to show someone when they ask what you learned.

How this bites you: you used `now(100)` and are now hammering the Gateway ten times a second for a structure that changes monthly. You forgot `str()` around the browse result — it's a Java object, not a Python string.

Lab 10: UDTs

 40 min · Pain: 

Goal: define once, stamp many.

1. Tag Browser → right-click `Tags` → `New Tag > Data Type`. Name it `Motor`.
2. Add members:

Name	Type	Data Type
<code>Running</code>	Memory	Boolean
<code>Fault</code>	Memory	Boolean
<code>Speed</code>	Memory	Float
<code>SpeedSetpoint</code>	Memory	Float
<code>RunHours</code>	Memory	Float

3. On `Speed`, enable **History**, and add an **Alarm**: `High Speed`, mode `Above Setpoint`, setpoint `1800`, priority `High`. On `Fault`, add an alarm: mode `Equal / Bit State`, priority `Critical`.
4. Add a **UDT parameter** on the definition: `MotorName` (String). Use it in the alarm's **Display Path** as `Line 1 / {MotorName}`. This is why parameters exist.
5. Create instances: right-click `Line1` → `New Tag > Data Type Instance > Motor`. Name `Pump1`, set `MotorName = Pump 1`. Repeat for `Pump2`, `Pump3`.
6. Go back to the definition and **add a member** `Amps` (Memory, Float). Save.
7. Look at all three instances. They all have `Amps`.

 You changed one thing and three instances updated — and you can explain why that scales to five hundred.

How this bites you: you built a UDT with no parameters, so instances can't differ in any meaningful way. Or you put the alarms on the instances and adding one alarm is now an afternoon of clicking.

Then: point the Lab 8 pattern at motors. Build `Components/MotorCard` taking a `motorPath` param, and a repeater over `[default]Line1`.

Lab 11: Named Query to Table

 35 min · Pain: 

Needs a database connection. If you don't have one, skip to Lab 12 and come back — don't stall the whole plan on it.

1. Check **Config > Databases > Connections** for a connection name.
2. In **Tools > Database Query Browser**, make a table and some rows:

```
CREATE TABLE production_log (  
  id INT IDENTITY PRIMARY KEY,  
  ts DATETIME,  
  line_id INT,  
  units INT  
);
```

Insert twenty rows across a few days and two lines.

3. **Named Queries** → **New** → **Production/GetTotals**:
 - Database: your connection; Type: **Query**
 - Parameters: **startDate** (DateTime), **endDate** (DateTime)
 - SQL:

```
SELECT line_id, SUM(units) AS total  
FROM production_log  
WHERE ts >= :startDate AND ts < :endDate  
GROUP BY line_id  
ORDER BY line_id
```

- Use the **Testing** tab to run it with sample values. Fix it here, not in the view — the feedback loop is ten times faster.
4. New view **Pages/Reports**, root **Flex** column.
 5. Add two **Date Time Input** components, renamed **StartDate** and **EndDate**.
 6. Add a **Table**.
 7. Bind the Table's **props.data** with a **Query** binding to **Production/GetTotals**, binding **startDate** → **../StartDate.props.value** and **endDate** → **../EndDate.props.value**.
 8. Save, preview, change a date. The table re-queries.

🚩 The table reacts to the date pickers and you wrote zero Python.

How this bites you: you typed property paths by hand instead of using the picker. You built SQL by concatenating strings (go back and read 6.3). Or the table is blank with no error — check the return format, it's the dataset-vs-array thing.

Lab 12: History and a Power Chart

🕒 30 min + wait · Pain: 🌶️🌶️

1. Make a tag actually move. Either add a **device simulator** connection in **Config > Device Connections** and create OPC tags from it, **or** create an Expression tag:

```
sin(getSecond(now(1000)) / 10.0) * 50 + 500
```

with history enabled. Crude, but it produces a real curve.

2. On the tag, open **History**: enable it, pick your storage provider, `Sample Mode = On Change`, `Deadband = 1`, `Max Time Between Samples = 1 minute`.
3. **Let it run ten minutes while you do something else.** Go get coffee; this is the one lab where waiting is the work.
4. New view `Pages/Trends`. Drop a **Power Chart**.
5. In Preview mode, use the Power Chart's own pen-selection UI to add your tag. Notice how much you got for one drag — range selection, multiple axes, export.
6. Now the developer-controlled way: drop a **Time Series Chart**, bind its data with a **Tag History** binding, pick your tag, range "realtime, last 1 hour", aggregation `Average`.

🚩 A curve with real history behind it, and you know the difference between "operator picks the pens" (Power Chart) and "developer picks the pens" (Time Series Chart).

How this bites you: you expected history to exist for the hour before you enabled it. Your deadband is so wide nothing got stored. You never set a historian provider on the tag.

Lab 13: Buttons That Do Things

🕒 30 min · Pain: 🌶️🌶️

1. On `Components/MotorCard` (or `TankCard`), add a **Button**, `props.text = "Start"`.
2. Event Configuration → `onActionPerformed` → **Script**:

```
def runAction(self, event):
    path = self.view.params.motorPath
    system.tag.writeBlocking([path + "/Running"], [True])
```

3. Add a "Stop" button writing `False`.
4. Preview, click, watch the tag in the Tag Browser.
5. **Now refactor it properly.** Project Library:

```
def setRunning(motorPath, running):
    result = system.tag.writeBlocking([motorPath + "/Running"], [bool(running)])
    return result[0].good
```

and the button becomes:

```
def runAction(self, event):
    plant.motors.setRunning(self.view.params.motorPath, True)
```

6. Add a third button with **no script at all**: a **Set Property** action writing `True` to `view.custom.showDetails`. Bind some component's `meta.visible` to that custom property.
7. In `Tools > Script Console`, test the library function directly:

```
plant.motors.setRunning("[default]Line1/Pump1", True)
```



Your button script is one line and your logic is testable without clicking anything.

How this bites you: you dropped the `def runAction(self, event):` wrapper. You used the old `system.tag.write` instead of `writeBlocking`. You did a read right after the write and got the old value — that's the Stale Read, and now you've met it in person.

Lab 14: Popups with Parameters



30 min · Pain: 🌶️🌶️

1. Create `Views/Popups/MotorFaceplate`, root **Flex**, param `motorPath` (input).
2. Build it with indirect bindings, exactly like Lab 8: speed, setpoint entry, run hours, start/stop.
3. Set the view's `props.defaultSize` to something popup-shaped — 420 × 520.
4. On `MotorCard`'s root container: Event Configuration → `onClick` → **Popup** action:
 - View `Popups/MotorFaceplate`, **ID** `motorFaceplate`
 - Params: `motorPath` → bind to `view.params.motorPath`
 - Title: an expression using the motor name; Modal: true; Draggable: true
5. Add a Close button inside the faceplate: **Popup > Close**, ID `motorFaceplate`.
6. Preview. Click three different cards. Each opens the faceplate for *that* motor.



One popup view serves every motor in the plant.

How this bites you: you gave each card a different popup `id`, so popups stack up. You passed a literal string instead of binding the param, so every card opens Pump 1. You made it non-modal and lost it behind the page.

Lab 15: Message Handlers



25 min · Pain: 🌶️🌶️🌶️

1. In `Docks/LeftNav`, add a Button "Refresh Data" → `onActionPerformed` → **Script**:

```
def runAction(self, event):
    system.perspective.sendMessage("refreshData", payload={}, scope="page")
```

2. In `Pages/Overview`, select the root container → Event Configuration → **Message Handlers** → add `refreshData`, scope **Page**:

```
def onMessageReceived(self, payload):
    self.getChild("FlexRepeater").refreshBinding("props.instances")
    system.perspective.print("Overview refreshed")
```

(Adjust the child path to match where your repeater actually sits.)

3. Preview the **full session**, not the single view — you need the dock to exist. Click Refresh. Check the Output Console for the print.

 A button in one view causes an action in a completely different view, with no reference between them.

How this bites you: scope mismatch — you sent `page` and listened on `view`. You tested in single-view preview where the dock doesn't exist. You went looking for the print in the browser console; it's on the Gateway, because [the server has the brain](#).

Lab 16: Alarms End to End

 30 min · Pain: 

1. Add an alarm to `Line1/Tank1/LevelPercent`: `High Level`, mode `Above Setpoint`, setpoint `90`, priority `High`, **Time On Delay** 5 seconds, **Display Path** `Tank 1 High Level`.
2. New view `Docks/AlarmBanner`, root Flex, with an **Alarm Status Table**. Filter to priority `High` and above, turn off columns you don't need, make it compact.
3. Add it as a **Bottom** dock in Shared Settings, ID `alarmBanner`, size 120.
4. Set `Tank1/Level` to `980`. Wait five seconds (that's your Time On Delay earning its keep).
5. The alarm appears. **Acknowledge** it. Set `Level` back to `450` and watch it clear.
6. Create `Pages/AlarmHistory` with an **Alarm Journal Table** and a date range. Your event should be in there.

 You caused an alarm, saw it, acked it, cleared it, and found it in history.

How this bites you: the journal table is empty because no alarm journal profile is configured (`Config > Alarming > Journal`). You skipped the time delay and now have a chattering alarm at the threshold. You left Display Path blank and your table shows raw tag paths.

Lab 17: Style Classes and Theming

 35 min · Pain: 

1. `Perspective > Styles` → New Style Class. Make a folder `status` with `running`, `stopped`, `fault`, `warning`. Set `backgroundColor`, `color`, `borderRadius`, `padding` in each.
2. Make `card/base`: padding, border, radius, a subtle shadow.
3. Go to `MotorCard` and **delete every inline background color** you set earlier.

- On the root container, set `props.style.classes = card.base`.
- Drive the status color from state, in **two steps** — this is the version I'd ship:
 - On the root, add `custom.state` (String), bound with an **Indirect Tag binding** to `{1}/Status`.
 - Bind the status Label's `props.style.classes` with a **Property** binding to `view.custom.state`, then add a **Map** transform: `"FAULT" → "status.fault"`, `"RUNNING" → "status.running"`, `"STOPPED" → "status.stopped"`, plus a fallback.

⚠ You **cannot** nest a property reference inside a tag reference — `{[default]{view.params.path}/Fault}` is not valid expression syntax. That's precisely what indirect bindings are for. When you catch yourself trying to assemble a tag path inside `{ }`, stop: you want an indirect binding.

- Change `status.fault`'s color in the Styles editor. Every card everywhere changes. That's the payoff.
- Add a theme toggle button:

```
def runAction(self, event):
    current = self.session.props.theme
    self.session.props.theme = "dark" if current != "dark" else "light"
```

Click it and check your screens still read in both themes.

🚩 You can restyle the whole application from the Styles tree, and dark theme produces no black-on-black anywhere.

How this bites you: you named a class after a color. You hardcoded a white background that becomes unreadable in dark theme.

Lab 18: Lock It Down

🕒 30 min · Pain: 🌶🌶

- `Config > Security > Users, Roles`. Create `op1` with role `Operator` and `sup1` with role `Supervisor`.
- In `Project Properties > Perspective > Permissions`, require authentication for the project.
- On the Start/Stop Button in `MotorCard`: component **Permissions** → require security level `Authenticated/Roles/Supervisor`, not-permitted behavior **Disable** (or Hide).
- Also bind `meta.visible` on a supervisor-only section:

```
isAuthorized(false, "Authenticated/Roles/Supervisor")
```

- Now enforce it for real.** In the Tag Browser, open the `Motor` UDT's `Running` member and require Supervisor on its write permissions.
- Open a session, log in as `op1`. The button is gone or disabled.
- Log in as `sup1`. It works.

8. Bonus: `Config > Security > Security Zones` — make a zone for your subnet and add it to the permission, so supervisor rights only apply from the control room.



The operator can't press it **and** couldn't write the tag even if they could press it.

How this bites you: you stopped at step 4 and called it secure — that's [the Hidden Button Fallacy](#). You tested as your own admin account and learned nothing.

Part 14. When It Breaks

Work down these in order. It's nearly always in the first five.

"My binding shows nothing"

1. Are you in **Preview Mode**?
2. Did you **save**?
3. Is the tag path right? Copy it from the Tag Browser instead of typing it.
4. Is the tag **quality** good? Look for the overlay.
5. Is the data the right **shape**? Table wants an array of objects; you may have a dataset.
6. **Open the binding dialog — it shows the live value.** That's your debugger and most people never notice it's there.

"My script doesn't run"

1. Is the signature exactly `def runAction(self, event): ?`
2. Check the **Output Console** and the Gateway logs (`Status > Logs`). Errors go there, not to the browser.
3. Put `system.perspective.print("here")` at the top to confirm it fires at all.
4. Test the logic in the **Script Console** in isolation.

"I wrote a tag but the screen shows the old value"

That's [the Stale Read](#). Don't read after write. Let the binding update. If you need confirmation the write was accepted, check `result[0].good`.

"Works in the Designer, not in the session"

1. Saved?
2. Session-only properties (`session.props.auth.*`) are empty or different in the Designer.
3. Permissions — you're an admin in the Designer; the operator isn't.
4. Design-time parameter defaults don't apply at runtime.

"The session is slow"

1. `Status > Perspective Sessions` — how many, and what's their load?
2. Count the bindings on the offending view. Repeaters multiply them.
3. Hunt for script transforms doing queries or tag reads.
4. Hunt for `now()` with a fast poll rate.
5. Hunt for Query bindings polling when a trigger would do.
6. `Status > Performance` and the logs for slow queries.

Where the logs are

Gateway web UI → **Status > Diagnostics > Logs** . Filter by logger name.

When you post on the Inductive Automation forum, this is the first thing anyone will ask for, so bring it unprompted: your version, the exact error, and the log excerpt. Do that and you'll get useful answers remarkably fast — it's an unusually good forum.

Part 15. The Rules

The laws from each Part, collected. If you print one page of this guide, print this one.

The laws

1. **The Law of the Server's Brain** — the browser is a screen; the Gateway holds state and runs logic.
2. **The 200-Motor Test** — before building anything twice, ask whether it survives two hundred copies. UDTs.
3. **The Parameter Promise** — every view takes parameters; no view hardcodes a tag path.
4. **The Faceplate Pattern** — indirect tag binding + view parameter = one view for the whole plant.
5. **The No-Junk-Tags Rule** — display concerns belong in transforms, not new tags.
6. **The Nobody's-Looking Test** — must it be true with no session open? Tag or Gateway event. Otherwise, binding.
7. **Configured over clever** — prefer actions to scripts; visible behavior beats hidden behavior.
8. **The Three-Line Rule** — event scripts validate, call the library, handle the result.
9. **Broadcast, don't reach** — views talk by message, not by poking each other's properties.
10. **No string SQL. Ever.** Named Queries only.
11. **Shared Settings, once** — docks configured in one place, not pasted per page.
12. **Navigate by page** — URLs, bookmarks and the back button are features.
13. **The Bindable Class List** — bind `props.style.classes`; never inline colors.
14. **The Hidden Button Fallacy** — hide it *and* enforce it on the tag and the query.

The traps

15. **The Stale Read** — never read straight after a write.
16. **The Coordinate Trap** — Flex by default; Coordinate only for pictures of physical things.
17. **`now()` without an argument** — you just built a screen that never stops working.
18. **Script transforms doing I/O** — multiply by sessions before you write one.
19. **History isn't retroactive** — enable it before you need it.
20. **Dataset vs array** — the blank table with no error is always this.

The habits

21. Export a view before a risky change. Ten seconds.
22. Test on the actual target device, early.
23. Set Display Path and Associated Data on every alarm.
24. Turn on auditing before you need it.
25. Name things for meaning, never appearance or position.
26. Read the manual page for the component you're using. Guessing is slower.

27. One screen, one job. The "everything" screen is always the one operators hate.

28. When stuck, rebuild it in a throwaway view. Half the time you find the cause while stripping it down.

Part 16. Cheat Sheets

Expression functions you'll use weekly

```
// logic
if(cond, a, b)                case(value, m1, r1, m2, r2, default)
coalesce(a, b)                binEnum(...)

// math
round(x, places)    abs(x)    max(a,b)    min(a,b)    sqrt(x)    pow(x,y)

// strings
concat(a, b, ...)    len(s)    substring(s, i, j)    replace(s, old, new)
toStr(x)              toInt(x) toFloat(x)            lower(s)    upper(s)
split(s, delim)       indexOf(s, sub)                trim(s)

// dates
now(pollRateMs)       dateFormat(d, "yyyy-MM-dd HH:mm:ss")
dateArithmetic(d, n, "hour")    dateDiff(d1, d2, "minute")
getHour24(d)            getDayOfWeek(d)    addDays(d, n)

// tags & security
tag("[default]Path/To/Tag")      isAuthorized(isAllOf, "Role")
hasRole("Supervisor")            runScript("plant.util.f", pollRate, arg)

// datasets
lookup(dataset, lookupValue, noMatchValue, lookupCol, resultCol)
toDataSet(...)    getKey(ds, row, col)    numRows(ds)    numCols(ds)
```

system.perspective.* — the ones that matter

```
system.perspective.navigate(page="/line/3")
system.perspective.navigate(url="https://example.com", newTab=True)
system.perspective.navigate(view="Pages/Detail", params={"id": 3})

system.perspective.openPopup(id, view, params={}, title="", modal=True,
                             draggable=True, resizable=False, showCloseIcon=True)
system.perspective.closePopup(id)
system.perspective.togglePopup(id, view, ...)

system.perspective.openDock(id)
system.perspective.closeDock(id)
system.perspective.toggleDock(id)

system.perspective.sendMessage(messageType, payload={}, scope="page")

system.perspective.print(message)    # -> Gateway logs / Output Console
system.perspective.refresh()
system.perspective.download(filename, data)
system.perspective.setTheme("dark")
system.perspective.login() / logout()
system.perspective.isAuthorized(isAllOf, securityLevels)
```

```
system.perspective.getSessionInfo()
system.perspective.closeSession(sessionId)
```

system.tag.*

```
qvs = system.tag.readBlocking(["[default]A", "[default]B"])  # LIST in, LIST out
value, quality, ts = qvs[0].value, qvs[0].quality, qvs[0].timestamp

results = system.tag.writeBlocking(["[default]A"], [42])
ok = results[0].good

system.tag.browse("[default]Line1", {"tagType": "AtomicTag"})
system.tag.configure(basePath, tagsList, collisionPolicy)
system.tag.queryTagHistory(paths=[...], startDate=..., endDate=..., returnSize=...)
```

system.db.*

```
system.db.runNamedQuery("Path/QueryName", {"param": value})
system.db.runPrepQuery("SELECT * FROM t WHERE id = ?", [id])      # if you must
system.db.runPrepUpdate("UPDATE t SET x = ? WHERE id = ?", [x, id])
system.db.runScalarQuery("SELECT COUNT(*) FROM t")
```

system.dataset.*

```
pyds = system.dataset.toPyDataSet(ds)
ds2 = system.dataset.addRow(ds, [v1, v2])
ds3 = system.dataset.filterColumns(ds, ["a", "b"])
system.dataset.toCSV(ds)
```

Property paths you'll type constantly

```
view.params.<name>           # view parameter
view.custom.<name>           # view-scoped variable
this.props.<name>            # this component
../Sibling.props.text        # relative reference in a binding
session.props.auth.userName
session.props.device.type
session.custom.<name>
page.props.pageId
```

Script object navigation

```
self                                # the component
self.parent
self.getSibling("Name")
self.getChild("Container").getChild("Label")
self.view
self.view.params.x
self.view.custom.y
self.session
self.page
self.refreshBinding("props.data")
```

Useful system tags

```
[System]Gateway/CurrentDateTime
[System]Gateway/SystemName
[System]Gateway/Performance/CPU Usage
[System]Gateway/Performance/Memory Usage
[System]Gateway/Network/Hostname
```


Part 17. Glossary

Term	What it actually means
Binding	A live wire from data to a component property
Breakpoint Container	Swaps between two child layouts at a pixel width
Coordinate Container	Positions children by x/y/w/h — for drawing physical things
Dataset	Ignition's native table type; what SQL and history hand back
Designer	The desk you sit at; edits projects live on the Gateway
Dock	A view attached to a page edge — nav, header, alarm banner
Drawing Editor	8.3's built-in vector illustration tool
Event Stream	8.3 resource: source → transform → handler pipeline
Flex Container	CSS-flexbox-style container; your default
Flex Repeater	N copies of one view, from an array of parameter objects
Gateway	The server. The brain. Runs everything, holds every value
IdP	Identity Provider — where users authenticate
Indirect tag binding	A tag binding whose path contains <code>{1}</code> -style placeholders
Jython	Python 2.7 on the JVM. No numpy
Named Query	Stored, parameterized, permission-checked SQL. The only kind you write
Page Configuration	Where URLs map to views, and docks get set up
Project Library	Project-scoped Python modules, callable from anywhere
Quality	Metadata on every tag value: Good / Bad / Uncertain, and why
Security Level	Hierarchical permission node, e.g. <code>Authenticated/Roles/Supervisor</code>
Security Zone	Permission based on <i>where</i> the request came from
Session	One user's running instance of the project. A window with no brain
Style Class	A named, reusable set of CSS properties
Tag	A named value with quality, timestamp, alarms, history
Tag Provider	A namespace of tags — <code>default</code> , <code>System</code>
Theme	Gateway-level palette: light, dark, custom
Transform	A step reshaping a binding's value: Map, Format, Expression, Script
UDT	User Defined Type — a reusable tag structure. Your biggest lever
View	The unit of Perspective design. Reusable, parameterized, embeddable
Vision	The older Java desktop module. Supported, not where new work goes

Part 18. Final Exam

Closed book. If you can't answer one, the link is your homework — no shame in it, that's what the links are for.

Architecture

1. Where does a Perspective event script execute, and name two consequences. → [1.2](#)
2. Trace a click from the operator's finger to the PLC. → [1.5](#)
3. Perspective or Vision — how do you tell which tutorial you're reading? → [1.3](#)

Layout 4. Header, scrollable body, footer. Which container, which position props? → [3.2](#) 5. When is Coordinate genuinely the right answer? → [3.2](#)

Bindings 6. One screen must serve two hundred motors. Describe the mechanism by name. → [Lab 8](#) 7. Name the four transform types and one use for each. → [4.3](#) 8. Expression binding or expression tag? What question decides it? → [4.5](#)

Reuse 9. What shape does `props.instances` want? → [3.4](#) 10. Why are UDTs the highest-leverage feature in the product? → [1.4](#)

Data 11. The non-convenience reason for Named Queries. → [6.3](#) 12. What's a Literal parameter and why is it dangerous? → [6.3](#) 13. Blank table, no error. First suspect? → [6.4](#)

Scripting 14. What's in scope inside `def runAction(self, event): ?` → [5.2](#) 15. Two views can't see each other. How do they talk? → [5.4](#) 16. Why three lines? → [5.3](#)

Security 17. Name the fallacy and give the complete fix. → [9.2](#)

8.3 18. Three things 8.3 added, and why each matters. → [Part 11](#)

The real test

Can you build the [capstone](#) without looking at the labs? That's the only question whose answer is worth anything to an employer.

Where to Go Next

- **Inductive University** (inductiveuniversity.com) — free, official, short videos with a version selector. Set it to **8.3**. Pairs well with these labs; do the Perspective track.
- **The user manual** (docs.inductiveautomation.com , version selector → **8.3**) — the appendix has a page per component listing every property. Bookmark the Perspective section.
- **The forum** (forum.inductiveautomation.com) — unusually high signal for a vendor forum. Search first; when you post, bring your version, the exact error, and the log excerpt.
- **8.3 release notes** (inductiveautomation.com/downloads/releasenotes/8.3.x) — skim what changed between 8.3.0 and 8.3.9 once, so nothing surprises you twice.
- **Ignition Exchange** — community components you can import and read. Reading other people's views is a genuinely fast way to level up.

After the capstone: project inheritance for multi-site work, the Reporting module, alarm notification pipelines, Event Streams, MQTT/Sparkplug, and redundancy.

One Last Thing

This guide is accurate to 8.3 as I know it, but module availability differs per install and point releases move things around.

When the screen disagrees with this document, believe the screen — then go read the manual page for whatever surprised you.

That habit right there — being surprised, then going and finding out why, instead of working around it — is the actual skill. Everything else in these fifty pages is just vocabulary.

Now go break something on a dev Gateway.